

LogiRota MVP · Semana 1

Persistência e Modelagem de Dados · Daniel Queiroz Soares · branch Daniel-Soares-11032410168 · 08/10/2026

1. Objetivo da entrega

Até a Aula 08, pontos de entrega e ruas viviam em listas fixas dentro do `main.py` e se perdiam ao fechar o programa. A Semana 1 fecha essa lacuna: o banco de dados passa a ser a fonte de verdade. Pontos, vias e pedidos cadastrados numa execução aparecem na execução seguinte.

2. Decisão de arquitetura: PostgreSQL no Docker, com SQLite como alternativa

A apostila propõe SQLite (arquivo único, sem servidor). Mantive o mesmo esquema e o mesmo contrato do `banco.py` fornecido e acrescentei um segundo motor, escolhido por variável de ambiente:

Modo	Quando é usado	Por quê
PostgreSQL 16 (Docker)	DATABASE_URL definida (já vem no <code>docker-compose.yml</code>)	Simula uma aplicação que recebe múltiplas requisições: o Postgres aceita várias conexões e escritas simultâneas, e o SQLite trava o arquivo inteiro a cada escrita. Com Docker, o professor sobe o banco com um único comando, no Windows ou no macOS.
SQLite (<code>logirota.db</code>)	DATABASE_URL ausente	Garante que <code>python main.py</code> funcione em qualquer máquina, sem Docker e sem instalar nada (exatamente como na apostila).

O restante do programa não sabe qual motor está em uso: `banco.py` devolve sempre tuplas e listas simples, e nenhum detalhe de SQL vaza para o domínio.

3. Modelagem: as quatro tabelas

Tabela	Colunas	Pergunta de negócio que responde
pontos	nome (PK), bairro, x, y	Quais endereços de entrega existem e onde ficam?
ruas	origem, destino (PK composta, FK → pontos)	Quais pontos estão ligados por uma rua? (arestas do grafo)
pedidos	id (auto), destino (FK → pontos), status (padrão "pendente")	O que ainda falta entregar, e para onde?
rotas_calculadas	id, origem, sequencia, distancia_total, criada_em	Que rotas já foram planejadas? (criada agora e usada a partir da Semana 2)

O peso de cada rua **não** é gravado: ele é derivado de `Ponto.distancia_ate` quando o grafo é montado, então nunca fica desatualizado em relação às coordenadas.

4. Arquivos criados ou alterados

Arquivo	O que faz
---------	-----------

logirota/banco.py (novo)	Camada de persistência: inicializar, salvar_ponto, salvar_rua, salvar_pedido, listar_pontos, listar_ruas, listar_pedidos_pendentes, banco_vazio.
main.py (alterado)	Cria as tabelas, popula a malha inicial só se o banco estiver vazio, monta o GrafoLista a partir do banco e aceita --pedido "Nome do ponto" para cadastrar pedidos.
docker-compose.yml, Dockerfile	Serviço db (postgres:16-alpine, volume logirota_dados, healthcheck) e serviço app que roda o main.py contra ele.
requirements.txt	psycopg[binary], necessário só no modo PostgreSQL.
README.md, .gitignore	Como rodar nos três modos; logirota.db fica fora do Git.
ponto.py, grafo.py	Sem nenhuma alteração , conforme a rubrica.

5. Decisões que atendem a rubrica

- **Sem duplicação ao reiniciar.** A malha inicial só é gravada quando banco_vazio() é verdadeiro, e todo INSERT de ponto/rua usa ON CONFLICT DO NOTHING (aceito por SQLite e Postgres) como segunda proteção.
- **Conexões sempre fechadas.** conectar() é um gerenciador de contexto: dá commit ao final do with, rollback se houver erro e close sempre. (O with sqlite3.connect() puro só faz commit, não fecha.)
- **Persistência separada do domínio.** banco.py não importa ponto.py nem grafo.py. Converter tupla em Ponto é feito em main.py (carregar_pontos). Por isso salvar_ponto recebe (nome, bairro, x, y): Ponto não expõe x/y publicamente e não foi alterado.
- **Sem consulta redundante.** As ruas são lidas uma única vez por execução e reaproveitadas para montar o grafo e contar.

6. Teste demonstrável (prova de fogo)

```

docker compose up -d db
docker compose run --rm app                                # 1ª execução: grava a malha
docker compose run --rm app --pedido "Padaria Distrito"    # cadastra pedido #1
docker compose down && docker compose up -d db             # derruba e sobe o banco
docker compose run --rm app                                # pedido #1 continua lá

```

Saída real da última execução, depois de derrubar e subir o container:

```

malha carregada de PostgreSQL (Docker).
8 pontos e 8 ruas no banco.
pedidos pendentes (1):
  #1 Padaria Distrito (pendente)

```

Verificação	Resultado
SQLite: 3 execuções seguidas	8 pontos e 8 ruas em todas (sem duplicar); pedido cadastrado na 2ª aparece na 3ª
PostgreSQL: down/up do container	Malha e pedido preservados no volume
Pedido para ponto inexistente	Mensagem com a lista de pontos válidos, sem erro

50 pedidos gravados por 20 threads simultâneas (Postgres)	50 ids únicos, nenhum pedido perdido
---	--------------------------------------

7. Próximo passo

A Semana 2 vai ler os pedidos pendentes de `listar_pedidos_pendentes()` e gravar o plano em `rotas_calculadas`, tabela que já existe.